Real-World Non-Embedded Use Cases: Smart Pointers

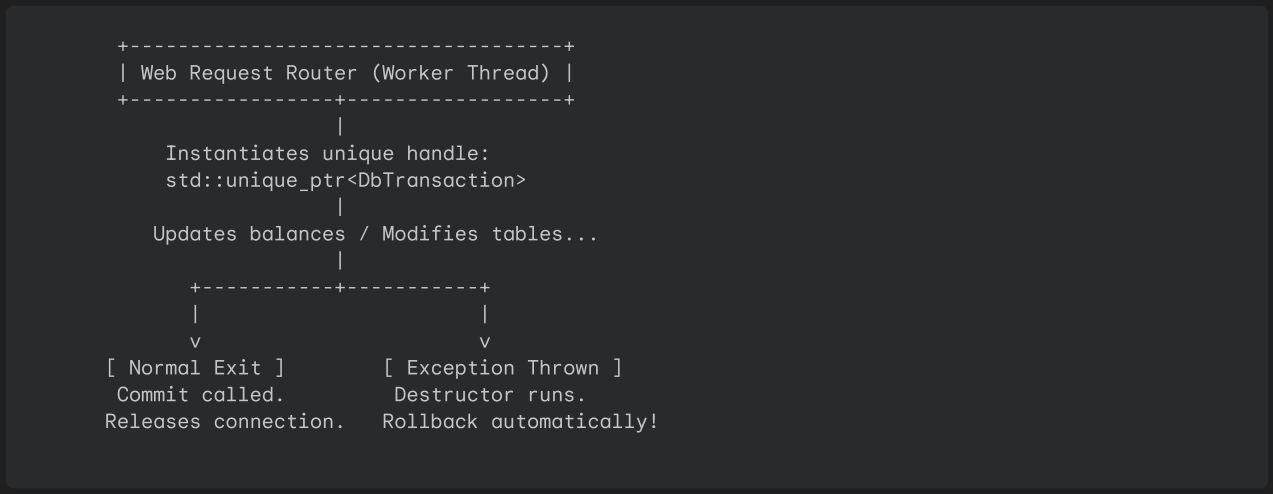
While embedded firmware operates under strict hardware constraints, standard application development (like desktop GUI apps, backend web servers, and game engines) faces different challenges: high concurrency, dynamic resource loading, and complex runtime lifecycles.

Below are three production-grade, non-embedded C++ patterns demonstrating how `std::unique_ptr`, `std::shared_ptr`, and `std::weak_ptr` manage system resources, optimize memory caches, and prevent dynamic application crashes.

1. `std::unique_ptr` Use Case: Scoped Database Transaction Handle (Backend Web APIs)

Imagine a high-throughput enterprise backend API server (like a gaming matchmaker or financial backend). When a web request comes in to modify user data, the server must:

1. **Acquire** a connection from a database pool.
2. **Begin** an atomic SQL transaction.
3. **Commit** the transaction if everything succeeds.
4. **Rollback** and cleanly release the connection back to the pool if *any* error, early return, or exception occurs during execution.



The Architectural Problem

If you manage this database transaction with a raw pointer, you run a massive risk of **resource leaks** and **data corruption**:

- If the function returns early due to a validation error, you must remember to manually write `delete transaction;`. If you forget, the database connection is held hostage indefinitely, eventually starving the server connection pool.
- If a sub-expression throws a runtime exception, the execution pointer skips right past your manual cleanup statements, leaving the transaction open and un-rolled-back in the SQL database.

The Modern C++ Solution

We use `std::unique_ptr<DbTransaction>` using the RAII pattern.

The `DbTransaction` wrapper's destructor is written to automatically execute a `ROLLBACK` in the database *unless* a manual `commit()` flag is explicitly set. Because a `std::unique_ptr` is strictly scoped and cannot be copied, C++ guarantees that the connection is cleanly closed or returned to the pool the exact nanosecond the scope exits, regardless of how it exits (successful completion, early return, or crash-inducing exceptions).

Code Implementation

```
#include <iostream>
#include <memory>
#include <string>
#include <stdexcept>

// Mock Database Connection
class DbConnection {
public:
    void executeQuery(const std::string& query) {
        std::cout << "  [SQL] Executing: " << query << "\n";
```

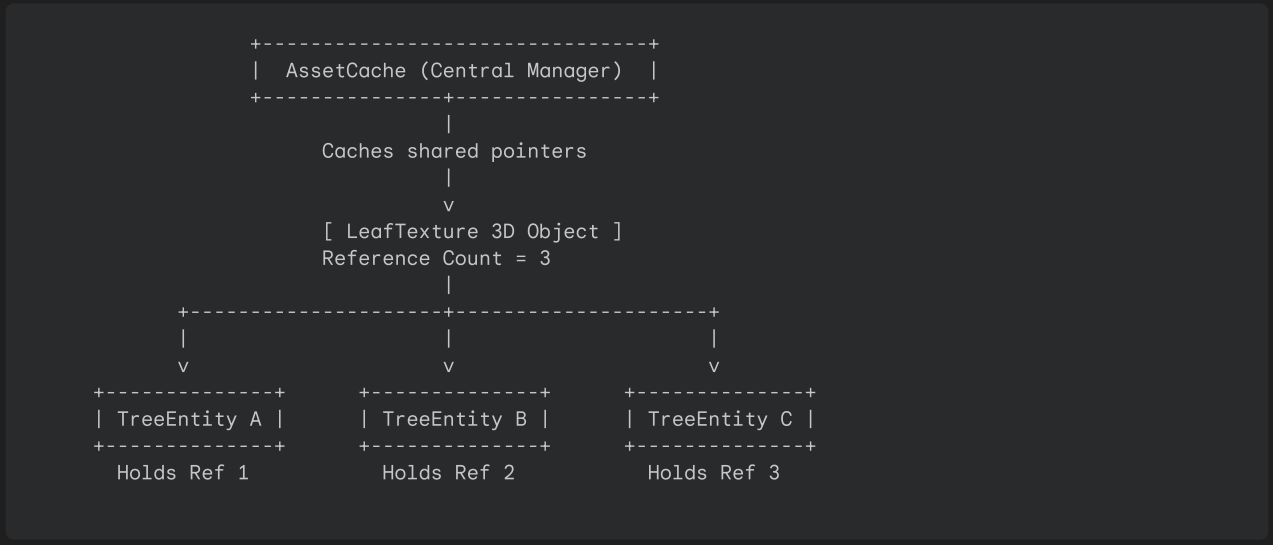
```
    }
    void rollback() {
        std::cout << "  [DB ENGINE] Rollback executed! All database changes reverted.\n";
    }
    void commit() {
        std::cout << "  [DB ENGINE] Transaction Committed successfully to SSD storage.\n";
    }
};

// Scoped Transaction Wrapper
class DbTransaction {
private:
    DbConnection* m_Conn;
    bool m_Committed;

public:
    explicit DbTransaction(DbConnection* conn) : m_Conn(conn), m_Committed(false) {
        std::cout << "[DbTransaction] Transaction context started.\n";
    }

    // RAII Safety Net: Executed automatically on exit
    ~DbTransaction() {
        if (!m_Committed && m_Conn) {
            std::cout << "[DbTransaction] WARNING: Scope exited without commit! Cleaning up...\n";
            m_Conn->rollback();
        }
        std::cout << "[DbTransaction] Context destroyed, releasing connection pointer.\n";
    }

    void commit() {
        if (m_Conn) {
            m_Conn->commit();
            m_Committed = true;
        }
    }

    // Force non-copyability to prevent duplicate transaction handlers
    DbTransaction(const DbTransaction&) = delete;
    DbTransaction& operator=(const DbTransaction&) = delete;
};

// Simulated Backend API endpoint
void processUserCheckout(DbConnection* conn, int cartValue, int userBalance) {
    std::cout << "--- Checkout API Started ---\n";

    // Allocate the transaction exclusively on the stack wrapped in a unique_ptr
    auto transaction = std::make_unique<DbTransaction>(conn);

    conn->executeQuery("UPDATE users SET status = 'checking_out'");

    // Simulate database calculations...
    if (userBalance < cartValue) {
        std::cout << "  [API Validation] Fail: Insufficient funds!\n";
        return; // Early return! unique_ptr goes out of scope and TRIGGERS DbTransaction Destructor ->
    }

    conn->executeQuery("SUBTRACT " + std::to_string(cartValue) + " FROM balances");

    // Simulate a sudden runtime crash (unhandled parsing exception)
    if (cartValue < 0) {
        throw std::runtime_error("Parsing error: Malformed checkout request payload!");
    }

    conn->executeQuery("UPDATE inventory SET stock = stock - 1");

    // Explicitly commit
    transaction->commit();
    std::cout << "--- Checkout API Successful ---\n\n";
}

int main() {
    DbConnection mockDb;

    std::cout << "=== SCENARIO A: Early Return (Insufficient Funds) ===\n";
    processUserCheckout(&mockDb, 500, 100);

    std::cout << "\n=== SCENARIO B: Exception Thrown (Malformed Data) ===\n";
    try {
```

```
        processUserCheckout(&mockDb, -50, 1000);
    } catch (const std::exception& e) {
        std::cout << "  [Main Process Handler] Caught Exception: " << e.what() << "\n";
    }

    return 0;
}
```

2. `std::shared_ptr` Use Case: Game Engine Global Asset Cache

In professional game engine architectures (like Unreal Engine or custom desktop engines), loading heavy assets (like $4K$ resolution textures, high-poly 3D models, or high-fidelity audio files) from SSD disk into RAM is incredibly slow.

To maintain a smooth frame budget, engines use an **Asset Cache**. If five different trees on a screen share the same leaf texture, the engine should only load that leaf texture *once* into RAM and share it.



The Architectural Problem

Game worlds are highly dynamic. Entities like trees, enemies, and UI screens are created and destroyed constantly.

- If we pass around raw pointers, how do we know when it is safe to delete the leaf texture? If Tree A is chopped down and deletes the texture, Trees B and C will instantly crash with a segfault on the next frame when they try to render.
- If we keep all textures loaded in memory forever, we will quickly run out of system RAM. We need a way to release assets from memory the exact moment they are no longer being rendered by any active entities in the scene.

The Modern C++ Solution

We use `std::shared_ptr<Texture>`.

The central `AssetCache` hands out shared pointers to every rendering entity that requests a specific file. As long as at least one active entity exists, the texture is kept in memory. The moment the last tree using that texture is destroyed, the texture's reference count drops to 0, and C++ automatically frees the texture memory from RAM.

Code Implementation

```
#include <iostream>
#include <memory>
#include <string>
#include <unordered_map>

struct Texture {
    std::string filePath;
    size_t sizeInBytes;

    Texture(std::string path, size_t size) : filePath(path), sizeInBytes(size) {
        std::cout << "  [ALLOC] " << filePath << " loaded into RAM (" << sizeInBytes / (1024 * 1024) <<
    }
    ~Texture() {
        std::cout << "  [FREE] " << filePath << " evicted from memory.\n";
    }
};
```

```
// Game Engine Asset Manager
class AssetCache {
private:
    std::unordered_map<std::string, std::shared_ptr<Texture>> m_Cache;

public:
    std::shared_ptr<Texture> getTexture(const std::string& path) {
        // If already cached, return the existing shared pointer (increments reference count)
        if (m_Cache.find(path) != m_Cache.end()) {
            std::cout << "  [Cache Hit] Serving: " << path << "\n";
            return m_Cache[path];
        }

        // Cache miss: Load from disk, save in cache, and return
        auto newTexture = std::make_shared<Texture>(path, 16 * 1024 * 1024); // mock 16MB
        m_Cache[path] = newTexture;
        return newTexture;
    }

    void purgeUnused() {
        std::cout << "[Cache Audit] Checking for unused assets...\n";
        auto it = m_Cache.begin();
        while (it != m_Cache.end()) {
            // If use_count is 1, it means the ONLY reference is the cache itself!
            // Nobody else is rendering it, so we can evict it.
            if (it->second.use_count() == 1) {
                std::cout << "  -> Evicting unused asset: " << it->first << "\n";
                it = m_Cache.erase(it);
            } else {
                ++it;
            }
        }
    }
};

// A renderable game entity
class RenderEntity {
private:
    std::string m_Name;
    std::shared_ptr<Texture> m_Texture; // Shared ownership

public:
    RenderEntity(std::string name, std::shared_ptr<Texture> tex) : m_Name(name), m_Texture(tex) {
        std::cout << "  [Entity] " << m_Name << " created. (Texture Use Count: " << m_Texture.use_count
    }
    ~RenderEntity() {
        std::cout << "  [Entity] " << m_Name << " destroyed.\n";
    }
};

int main() {
    std::cout << "=== GAME ENGINE STARTUP ===\n";
    AssetCache cache;

    // 1. Spawning entities that share the same texture
    std::cout << "\n--- Spawning Game Entities ---\n";
    auto texTree = cache.getTexture("textures/bark.png");

    auto* tree1 = new RenderEntity("Tree_Alpha", texTree);
    auto* tree2 = new RenderEntity("Tree_Beta", texTree);

    // 2. Load separate GUI layout asset
    auto texGUI = cache.getTexture("textures/ui_panel.png");
    auto* hpBar = new RenderEntity("HealthBar", texGUI);

    // 3. Destroy Trees (Eviction Check)
    std::cout << "\n--- Destroying Game Entities ---\n";
    delete tree1;
    delete tree2;

    // Main pointer dropped in main scope
    texTree.reset();

    // Audit Cache: Bark is no longer used, ui_panel is still held by hpBar
    cache.purgeUnused();

    std::cout << "\n--- Releasing UI ---\n";
```

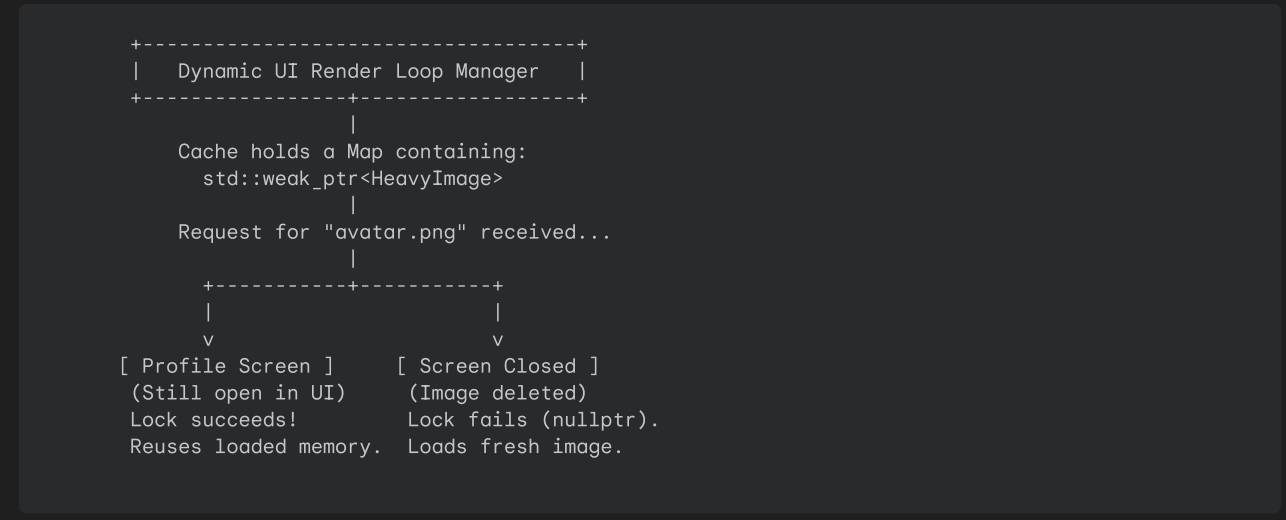
```
    delete hpBar;

    std::cout << "\n=== GAME ENGINE SHUTDOWN ===\n";
    return 0;
}
```

3. `std::weak_ptr` Use Case: Non-Owning Image Cache with Auto-Eviction

In desktop image editing applications (like photo managers or dynamic UI dashboards loading avatars), images must be cached to prevent stuttering as users browse through tabs.

To manage RAM correctly, we want an asset cache that:

1. Hands out loaded images to active UI rendering windows.
2. Automatically evicts the image from memory if **no active window** is currently displaying it.
3. Does **not** keep images alive just because they have an entry in the Cache table.



The Architectural Problem

If the central caching table holds `std::shared_ptr`, the image's reference count will **always be at least 1** (because the cache table itself holds a reference). The image will never be deleted, resulting in a classic desktop **memory leak** where memory usage climbs continuously as the user browses through files.

If the cache uses a raw pointer, and a UI window gets closed and deletes the image, the cache will hold a **dangling pointer**. On the next request, the cache will return invalid memory, causing an instant desktop application crash.

The Modern C++ Solution

The cache stores `std::weak_ptr<HeavyImage>`.

- When a window requests an image, the cache attempts to call `.lock()`.
- If a UI window is already keeping the image alive, `.lock()` succeeds, returning a temporary `shared_ptr` (zero disk overhead, maximum rendering speed).
- If the user closed all windows using that image, `.lock()` returns `nullptr`. The cache detects that the asset is dead, purges the stale entry, and safely loads a fresh copy from the hard drive without any danger of reading corrupted memory.

Code Implementation

```
#include <iostream>
#include <memory>
#include <string>
#include <unordered_map>

struct HeavyImage {
    std::string name;
    HeavyImage(std::string n) : name(n) {
        std::cout << "  [HDD] Reading image: " << name << " from hard drive.\n";
    }
    ~HeavyImage() {
        std::cout << "  [MEM] Destructor: eviction of image data for: " << name << "\n";
    }
```

```
};

class ImageCache {
private:
    std::unordered_map<std::string, std::weak_ptr<HeavyImage>> m_CacheTable;

public:
    std::shared_ptr<HeavyImage> retrieve(const std::string& imageName) {
        auto it = m_CacheTable.find(imageName);

        if (it != m_CacheTable.end()) {
            // Attempt to promote the weak pointer
            if (auto sharedImage = it->second.lock()) {
                std::cout << "  [Cache Hit] Image '" << imageName << "' is already in RAM!\n";
                return sharedImage;
            } else {
                // The image was freed in the background. Clean up cache entry.
                std::cout << "  [Cache Stale] Image '" << imageName << "' was evicted. Reloading...\n";
            }
        }

        // Load new image and cache a weak reference to it
        auto newImage = std::make_shared<HeavyImage>(imageName);
        m_CacheTable[imageName] = newImage; // Automatically downgrades to weak_ptr inside map
        return newImage;
    }
};

int main() {
    std::cout << "=== DESKTOP IMAGE VIEW BOOT ===\n\n";
    ImageCache engineCache;

    {
        std::cout << "1. User opens Profile Settings Screen...\n";
        std::shared_ptr<HeavyImage> profilePic = engineCache.retrieve("user_profile_99.png");

        std::cout << "\n2. User switches tabs, reloading Profile Pic...\n";
        std::shared_ptr<HeavyImage> profilePicDupe = engineCache.retrieve("user_profile_99.png");

        std::cout << "\nActive profile references: " << profilePic.use_count() << "\n";

        std::cout << "\n3. Closing profile page (Releasing active shared references)...\n";
    } // Both shared_ptrs go out of scope. RAM is immediately released!

    std::cout << "\n4. User navigates back to Profile Settings Page (Cache Eviction Audit)...\n";
    // System automatically detects the image is no longer in RAM, reloading from disk safely!
    std::shared_ptr<HeavyImage> profilePicReopened = engineCache.retrieve("user_profile_99.png");

    std::cout << "\n=== SYSTEM SHUTDOWN ===\n";
    return 0;
}
```

Summary Reference Table

Pointer	Who "Owns" the Resource?	Desktop/Server Use Case	Non-Embedded Safety Guarantee	Performance Overhead
`std::unique_ptr`	Exactly one module / scope.	Scoped Database Transaction, UI Input Focus Handle, TCP Socket Connection.	Eliminates manual cleanups and guarantees safety under unexpected exceptions.	**Zero.** Identical to standard pointer math.
`std::shared_ptr`	Multiple runtime entities.	Global System Configuration, Engine Texture, Shared Node inside a UI Graph.	Tracks multi-threaded lifetimes, preventing silent memory leaks and use-after-free crashes.	Small atomic reference tracking block on the heap.
`std::weak_ptr`	Nobody. A passive spectator.	Registry of event listeners, dynamic image cache keys, parent node links.	Eliminates dynamic memory circular lock traps and prevents dereferencing deleted references.	Negligible tracking overhead inside the control block.